

Resume Screening Automation System

Technical Source Code Documentation

Document Type:	Source Code Implementation & Architecture Reference
Developer:	Swetha Battula
Target Domain:	Natural Language Processing & HR Tech Automation
Repository:	https://github.com/SwethaBattula/resume-screening-automation-system
Status:	Functional Release / Hackathon Edition

2. Project Overview

Purpose: The Resume Screening Automation System automates candidate evaluation by parsing multi-format PDF resumes, extracting technical skills using NLP and regex phrase boundaries, analyzing professional experience, and scoring applicants against target Job Descriptions.

Problem Statement: Manual resume screening is labor-intensive, slow, subjective, and prone to human bias. HR teams processing high volumes of technical applicants face throughput bottlenecks and inconsistent evaluations.

Solution Overview: This system combines a modular Python processing engine with an interactive Streamlit presentation layer. It extracts text, preprocesses content via spaCy/NLTK, matches technical skills against a 240+ skill repository, computes a weighted score (80% Skill Match + 20% Experience Match), generates executive candidate summaries, and exports shortlists to CSV and JSON formats.

3. Objectives

- Automate PDF text extraction with pdfplumber layout parsing and PyPDF2 stream fallback.
- Implement NLP preprocessing including lowercasing, tokenization, stopword filtering, and lemmatization.
- Perform phrase-boundary technical skill matching using pre-compiled LRU-cached regex patterns.
- Implement contact-proximity candidate name extraction to eliminate header keyword false positives.

- Compute candidate scores using an extensible 80% skill + 20% experience weighted model.
- Provide a zero-pandas Streamlit recruiter dashboard for candidate ranking and profile inspection.
- Export standardized CSV and JSON screening reports for downstream HR integration.

4. Technology Stack

The system is built using modern Python libraries and specialized NLP toolkits.

Technology	Role & Category	Description
Python 3.11	Core Language	Primary backend implementation language.
Streamlit	Frontend Dashboard	Interactive recruiter dashboard presentation layer.
spaCy (en_core_web_sm)	NLP Engine	Tokenization, entity recognition, and lemmatization.
NLTK	NLP Utilities	Fallback stopwords processing and text normalization.
pdfplumber	PDF Extraction	Layout-aware PDF text extraction engine.
PyPDF2 / pypdf	PDF Fallback	Secondary PDF parser for unreadable streams.
Scikit-learn	ML & Analytics	Feature processing and similarity calculation utilities.

5. Project Architecture

The application follows a clean layered architecture separating presentation, orchestration, domain processing, and data access layers:

- **Presentation Layer (app.py):** Streamlit web dashboard built strictly with native layout components (st.columns, st.container, st.markdown, st.write, st.progress) avoiding pandas dependencies.
- **Orchestration Layer (utils/resume_pipeline.py):** Exposes batch process APIs (process_resume, process_resume_batch) coordinating all backend steps.
- **Text Extraction Layer (utils/extractor.py):** Multi-strategy PDF extractor using pdfplumber with PyPDF2 fallback.
- **NLP & Parsing Layer (utils/preprocess.py, utils/parser.py):** spaCy/NLTK pipeline, contact-proximity name extraction, email/phone regex, and date-range experience parsing.
- **Skill Matching Engine (utils/skills.py):** Dataset loader (240+ skills) and LRU-cached regex phrase-boundary matcher.

- **Scoring & Summary Engine (utils/scorer.py):** Extensible 80/20 scoring model, recommendation classifier, and 2-3 sentence executive candidate summary generator.
- **Export Layer (utils/exporter.py):** Formatter generating flat records, CSV strings, and JSON reports.

6. Project Folder Structure

```
ResumeScreeningAutomation/
|-- app.py                # Streamlit Presentation Layer
|-- main.py              # CLI Entry Point & Batch Execution Demo
|-- config.py            # Centralized Configuration & Scoring Weights
|-- utils/               # Backend Modular Engine
|   |-- __init__.py      # Package Initialization
|   |-- extractor.py     # PDF Text Extraction (pdfplumber + PyPDF2 fallback)
|   |-- preprocess.py    # spaCy / NLTK Text Preprocessing Engine
|   |-- parser.py        # Resume Contact, Education, & Experience Parser
|   |-- skills.py        # LRU-cached Technical Skill Matcher
|   |-- scorer.py        # Weighted Match Scorer & Summary Generator
|   |-- resume_pipeline.py # Master Pipeline Orchestrator
|   |-- exporter.py      # CSV / JSON Exporter & Record Formatter
|   |-- logger.py        # Centralized Logging System
|-- data/                # Datasets & Benchmark Job Description
|   |-- skills.txt       # Categorized Skills Database (240+ skills)
|   |-- sample_job_description.txt # Benchmark Job Description
|-- tests/               # Automated Test Suite
|   |-- test_backend.py  # 18 Edge-Case Unit Tests
|-- output/              # Exported Screening Results Directory
|-- requirements.txt      # Package Dependencies Specification
|-- README.md            # Repository Overview & User Guide
```

7. Module Description

app.py: Streamlit presentation layer. Renders recruiter ranking cards, match progress bars, initials avatars, detailed profile tabs, and CSV/JSON download buttons without importing pandas.

main.py: CLI driver script. Loads configuration, executes batch resume processing against benchmark Job Description, prints formatted rankings, and exports reports.

config.py: Centralized configuration module. Stores scoring weights (SKILL_WEIGHT=0.80, EXP_WEIGHT=0.20), thresholds (80% Shortlisted, 60% Consider), paths, and logging formats.

utils/extractor.py: Handles PDF text extraction. Tries pdfplumber first to preserve visual layouts, falling back to PyPDF2 if stream objects are corrupted.

utils/preprocess.py: NLP preprocessing pipeline. Performs lowercasing, punctuation removal, stopwords filtering, tokenization, and spaCy/NLTK lemmatization.

utils/parser.py: Extracts structured candidate profile data: Email regex, Phone regex, Education degree detection, date-range experience calculation, and contact-proximity candidate name extraction.

utils/skills.py: Loads skills.txt dataset into memory and uses @lru_cache regex compilation to perform word-boundary and multi-word phrase technical skill matching.

utils/scorer.py: Calculates weighted match scores, assigns recommendations (Shortlisted/Consider/Rejected), calculates summary metrics, and generates 2-3 sentence executive candidate summaries.

utils/resume_pipeline.py: Exposes process_resume() and process_resume_batch() to coordinate text extraction, parsing, scoring, and output generation.

utils/exporter.py: Converts candidate results into flat dictionaries, tabular display records, CSV strings/files, and JSON strings/files.

8. Resume Screening Workflow

The system executes a sequential pipeline for candidate screening:

1. Upload Job Description (.txt file or raw text input)
2. Upload Multiple PDF Resumes
3. Multi-Strategy PDF Text Extraction (pdfplumber primary, PyPDF2 fallback)
4. NLP Preprocessing (lowercasing, punctuation removal, stopwords filtering, lemmatization)
5. Candidate Information Parsing (Name, Email, Phone, Education, Experience)
6. Technical Skill Extraction (regex boundary matching against skills database)
7. Focused JD Skill Extraction (prioritizes explicit required skills sections)
8. Skill Match Calculation (Candidate matched skills vs JD required skills)
9. Experience Analysis (Explicit experience mentions & parsed date-range calculation)
10. Weighted Candidate Scoring (80% Skill Match + 20% Experience Match)
11. Recommendation Classification & Executive Summary Generation
12. Candidate Ranking & Recruiter Dashboard Visualization
13. Multi-Format Report Export (CSV & JSON download buttons)

9. Candidate Scoring Algorithm

The candidate evaluation model uses an extensible weighted scoring framework:

- **Skill Match Score (80% Weight):** Computed as the percentage overlap between candidate extracted technical skills and JD required skills:

$$\text{Skill Score} = (\text{Matched Skills Count} / \text{Required JD Skills Count}) * 100$$

- **Experience Score (20% Weight):** Computed as candidate experience years relative to JD required experience years (capped at 100%):

$$\text{Exp Score} = \min(100.0, (\text{Candidate Exp} / \text{Required Exp}) * 100)$$

- **Final Score Formula:**

$$\text{Final Score} = (\text{Skill Score} * 0.80) + (\text{Experience Score} * 0.20)$$

- **Recommendation Boundaries:**

- **Shortlisted:** Final Score $\geq$ 80.0%
- **Consider:** $60.0\% \leq$ Final Score $<$ 80.0%
- **Rejected:** Final Score $<$ 60.0%

10. Frontend Features

The Streamlit dashboard (app.py) provides a modern presentation layer designed for HR recruiters:

- **Hero Header Banner:** Prominent title, project overview subtitle, and technology stack tags.
- **Sidebar Setup:** File uploaders for Job Description (.txt) and multiple PDF resumes, along with compact scoring criteria reference cards.
- **Dashboard Cards:** Summary metrics showing Total Candidates, Shortlisted, Consider, and Rejected counts.
- **Candidate Rankings Tab:** Recruiter-style candidate cards featuring rank numbers, CSS initials avatars, st.progress match bars, top 5 matched skills (+N more), and recommendation status badges.
- **Detailed Candidate Profile Tab:** Candidate selector dropdown displaying Executive Summary, Education, Matched Skills, Missing Required Skills, Scorecard metrics, and contact metadata.
- **Export Section:** Download buttons for CSV and JSON reports.

11. Error Handling

The system implements robust error handling across all pipeline stages:

- **Corrupted PDF Files:** Handled via try-except blocks; if pdfplumber fails, PyPDF2 fallback is attempted. If both fail, an error status is returned gracefully.
- **Empty Resume Files:** Detected by file size checks (0 bytes) and empty text validations, raising warnings without crashing batch processing.
- **Missing Job Description:** Checked by frontend and backend; default target experience (2.0 yrs) is used if required experience is unparsed.
- **Missing Candidate Details:** Name fallback hierarchy defaults to 'Unknown Candidate', missing emails/phones default to 'N/A'.

12. Testing & Validation

The codebase includes comprehensive test coverage and validation workflows:

- **Automated Test Suite (tests/test_backend.py):** Contains 18 unit and edge-case integration tests covering empty PDFs, corrupted PDFs, missing contact info, empty job descriptions, and large text inputs. Executed via *python -m unittest discover -s tests*.
- **CLI Execution Verification:** Executed via *python main.py* to verify batch execution against benchmark resumes.
- **Frontend Verification:** Verified via *python -m streamlit run app.py* ensuring zero pandas dependencies and clean UI rendering.

13. Screenshots Section

Below are designated placeholders for application screenshots:

13.1 Home Screen & Upload Interface

[Place screenshot image here]

13.2 Candidate Rankings Dashboard Cards

[Place screenshot image here]

13.3 Detailed Candidate Profile & Scorecard

[Place screenshot image here]

13.4 Export Reports Section (CSV & JSON)

[Place screenshot image here]

14. Future Enhancements

- **OCR Integration:** Add Tesseract/EasyOCR to parse image-based and scanned PDF resumes.
- **Semantic Vector Search:** Incorporate Sentence-Transformers / BERT embeddings for semantic skill matching beyond exact keyword matching.
- **LLM Assistance:** Integrate LLM APIs for qualitative resume synthesis and interview question generation.
- **ATS Integration:** Build REST APIs connecting with ATS platforms like Workday and Greenhouse.
- **Cloud Deployment:** Containerize application via Docker for AWS Elastic Container Service / Streamlit Cloud deployment.

15. GitHub Repository

The complete source code, test suite, datasets, and documentation are published on GitHub:

Repository Link: <https://github.com/SwethaBattula/resume-screening-automation-system>

Default Branch: main

16. Conclusion

The Resume Screening Automation System delivers a modular, reproducible, and scalable solution for technical resume screening. By combining multi-strategy PDF text extraction, spaCy/NLTK NLP preprocessing, phrase-boundary skill matching, weighted candidate scoring, and a clean Streamlit presentation layer, the application significantly reduces initial recruitment overhead while maintaining objective candidate rankings.